

PART 1: LLM vs Agent — The Critical Difference

What Makes an Agent an Agent

LLM (what you've been using): Input → [THINK] → Output (text)

"What's wrong with this log?" → "This looks like an OOMKilled error. You should increase memory limits." → THE LLM CANNOT ACTUALLY INCREASE THE MEMORY LIMITS

AGENT (what you're about to build): Input → [THINK] → [ACT] → [OBSERVE] → [THINK] → [ACT] → Output

"Fix the OOMKilled pod" → THINKS: "I need to check the current memory limits first" → ACTS: calls `get_pod_spec("payment-service")` → OBSERVES: memory limit is 256Mi, current usage peaks at 400Mi → THINKS: "The limit is too low. I should increase to 512Mi" → ACTS: calls `update_deployment_memory("payment-service", "512Mi")` → OBSERVES: deployment updated, pod restarting → RESPONDS: "I increased the memory limit from 256Mi to 512Mi. The pod is restarting. Monitor for 5 minutes to confirm stability."

Analogy: "You know the difference between a monitoring dashboard and a runbook automation system? The dashboard shows you data — you think, you decide, you act. The automation system observes metrics, decides what to do based on rules, and takes action — restart a pod, scale a service, open a ticket. An AI agent is like a runbook automation system, but instead of hardcoded rules, it uses an LLM to reason about what to do."

The Agent Loop

```
graph TD
    Input[Input] --> Observe[OBSERVE]
    Observe --> Think[THINK]
    Think --> Act[ACT]
    Act --> Execute[Execute]
    Execute --> LLM[LLM]
    LLM --> Receive[Receive]
    Receive --> Observe
    subgraph Loop
        Observe
        Think
        Act
        Execute
        LLM
        Receive
    end
    Loop --> Done{DONE?}
    Done -- Yes --> Response[return final response]
    Done -- No --> Loop
```

graph TD
 Input[Input] --> Observe[OBSERVE]
 Observe --> Think[THINK]
 Think --> Act[ACT]
 Act --> Execute[Execute]
 Execute --> LLM[LLM]
 LLM --> Receive[Receive]
 Receive --> Observe
 subgraph Loop
 Observe
 Think
 Act
 Execute
 LLM
 Receive
 end
 Loop --> Done{DONE?}
 Done -- Yes --> Response[return final response]
 Done -- No --> Loop

PART 2: Tool Calling — How It Actually Works

The Mechanics at the API Level

Step-by-step flow:

STEP 1: You define tools (JSON schema) 🔗 Tell the model what tools exist, what they do, what parameters they take

STEP 2: You send a message with tools attached 🔗 The model sees the tools and the user's message

STEP 3: The model decides to call a tool 🔗 Instead of returning text, it returns a "tool_use" block with the tool name and parameters

STEP 4: YOUR CODE executes the tool 🔗 The model doesn't execute anything. You run the function, call the API, query the database — whatever the tool does.

STEP 5: You send the tool result back to the model 🔗 The model sees what the tool returned

STEP 6: The model decides what to do next 🔗 Call another tool? Return a final response? The loop continues.

Implementation in Python (Raw API)

```
import anthropic
import json
```

```
client = anthropic.Anthropic()
```

```
# STEP 1: Define tools
```

```
tools = [
    {
        "name": "get_pod_status",
        "description": "Get the current status of a Kubernetes pod including its state, restart count, and resource usage. Use this when you need to check if a pod is healthy or diagnose pod issues.",
        "input_schema": {
            "type": "object",
```

```

    "properties": {
      "pod_name": {
        "type": "string",
        "description": "Name of the pod (e.g., 'payment-service-7d4b8c-x2k9p')"
      },
      "namespace": {
        "type": "string",
        "description": "Kubernetes namespace (default: 'production')",
        "default": "production"
      }
    },
    "required": ["pod_name"]
  },
  {
    "name": "get_pod_logs",
    "description": "Retrieve recent logs from a Kubernetes pod. Use this to investigate errors, crashes, or unexpected behavior. Returns the last N lines of logs.",
    "input_schema": {
      "type": "object",
      "properties": {
        "pod_name": {
          "type": "string",
          "description": "Name of the pod"
        },
        "namespace": {
          "type": "string",
          "default": "production"
        },
        "lines": {
          "type": "integer",
          "description": "Number of log lines to retrieve (default: 50)",
          "default": 50
        },
        "previous": {
          "type": "boolean",
          "description": "If true, get logs from the previous (crashed) container instance",
          "default": False
        }
      },
      "required": ["pod_name"]
    }
  },
  {
    "name": "get_recent_deployments",
    "description": "List recent deployments for a service in the last N hours. Use this to check if a recent code change might have caused an issue.",
    "input_schema": {
      "type": "object",
      "properties": {
        "service_name": {
          "type": "string",
          "description": "Name of the service (e.g., 'payment-service')"
        }
      }
    }
  }
}

```

```

        "hours": {
            "type": "integer",
            "description": "Look back this many hours (default: 24)",
            "default": 24
        },
    },
    "required": ["service_name"]
}
},
{
    "name": "create_incident_ticket",
    "description": "Create an incident ticket in ServiceNow. Use this when the issue requires tracking and team notification. IMPORTANT: Only create tickets for ERROR or CRITICAL issues.",
    "input_schema": {
        "type": "object",
        "properties": {
            "title": {
                "type": "string",
                "description": "Brief incident title"
            },
            "severity": {
                "type": "string",
                "enum": ["SEV1", "SEV2", "SEV3", "SEV4"],
                "description": "Incident severity"
            },
            "description": {
                "type": "string",
                "description": "Detailed incident description"
            },
            "service": {
                "type": "string",
                "description": "Affected service name"
            }
        },
        "required": ["title", "severity", "description", "service"]
    }
}
]

```

STEP 4: Mock tool implementations (in production, these call real APIs)

```

def execute_tool(tool_name: str, tool_input: dict) -> str:
    """Execute a tool and return the result as a string."""

```

```

    if tool_name == "get_pod_status":
        # Mock response — in production, this calls kubectl or K8s API
        return json.dumps({
            "pod": tool_input["pod_name"],
            "namespace": tool_input.get("namespace", "production"),
            "status": "CrashLoopBackOff",
            "restart_count": 5,
            "last_restart": "2 minutes ago",
            "exit_code": 137,
            "reason": "OOMKilled",
            "memory_limit": "256Mi",

```

```

        "memory_usage_peak": "412Mi",
        "cpu_usage": "45m"
    })

    elif tool_name == "get_pod_logs":
        return json.dumps({
            "pod": tool_input["pod_name"],
            "lines": [
                "2024-03-15 14:22:30 INFO Starting payment-service v2.4.1",
                "2024-03-15 14:22:31 INFO Connected to database",
                "2024-03-15 14:22:32 INFO Loading transaction cache into memory",
                "2024-03-15 14:22:33 WARN Memory usage at 85% of limit",
                "2024-03-15 14:22:34 ERROR java.lang.OutOfMemoryError: Java heap space",
                "2024-03-15 14:22:34 FATAL Process killed by OOM killer"
            ]
        })

    elif tool_name == "get_recent_deployments":
        return json.dumps({
            "service": tool_input["service_name"],
            "deployments": [
                {
                    "version": "v2.4.1",
                    "timestamp": "2024-03-15 13:45:00",
                    "deployer": "ci-pipeline",
                    "changes": "Added transaction caching for performance improvement"
                },
                {
                    "version": "v2.4.0",
                    "timestamp": "2024-03-14 10:00:00",
                    "deployer": "ci-pipeline",
                    "changes": "Bug fix for payment timeout handling"
                }
            ]
        })

    elif tool_name == "create_incident_ticket":
        return json.dumps({
            "ticket_id": "INC0012345",
            "status": "created",
            "url": "https://company.service-now.com/incident/INC0012345"
        })

    else:
        return json.dumps({"error": f"Unknown tool: {tool_name}"})

```

The Agent Loop Implementation

```

def run_agent(user_message: str, max_iterations: int = 10):
    """Run the agent loop with tool calling."""

    messages = [{"role": "user", "content": user_message}]

    system_prompt = """You are an SRE Agent that helps investigate and resolve

```

production incidents. You have access to tools for checking pod status, reading logs, checking deployments, and creating incident tickets.

APPROACH:

1. Start by gathering information (check status, logs, recent changes)
2. Analyze the evidence before drawing conclusions
3. Recommend or take action based on evidence
4. Create incident tickets for ERROR/CRITICAL issues

RULES:

- Always check pod status before making recommendations
- Check recent deployments to see if a change caused the issue
- Never create a ticket without first investigating
- Be specific in your analysis — cite exact data from tool results
- If you're unsure, say so rather than guessing""

```
iteration = 0
```

```
while iteration < max_iterations:
```

```
    iteration += 1
```

```
    print(f"\n--- Agent iteration {iteration} ---")
```

```
    # STEP 2 & 3: Send message, model may request tool calls
```

```
    response = client.messages.create(  
        model="claude-sonnet-4-20250514",  
        max_tokens=1500,  
        system=system_prompt,  
        tools=tools,  
        messages=messages  
    )
```

```
    # Check stop reason
```

```
    if response.stop_reason == "end_turn":
```

```
        # Model is done — extract final text response
```

```
        final_text = ""
```

```
        for block in response.content:
```

```
            if block.type == "text":
```

```
                final_text += block.text
```

```
        print(f"\n📄 Agent response:\n{final_text}")
```

```
        return final_text
```

```
    elif response.stop_reason == "tool_use":
```

```
        # Model wants to call tools
```

```
        # Add the assistant's response (which includes tool_use blocks)
```

```
        messages.append({"role": "assistant", "content": response.content})
```

```
    # Process each tool call
```

```
    tool_results = []
```

```
    for block in response.content:
```

```
        if block.type == "tool_use":
```

```
            tool_name = block.name
```

```
            tool_input = block.input
```

```
            tool_id = block.id
```

```

print(f" ☒ Calling: {tool_name}({json.dumps(tool_input)})")

# STEP 4: Execute the tool
result = execute_tool(tool_name, tool_input)
print(f" ☒ Result: {result[:200]}...")

tool_results.append({
    "type": "tool_result",
    "tool_use_id": tool_id,
    "content": result
})

# STEP 5: Send tool results back
messages.append({"role": "user", "content": tool_results})

else:
    print(f"Unexpected stop reason: {response.stop_reason}")
    break

print(f"☒ Agent reached max iterations ({max_iterations})")
return "Agent reached maximum iterations without completing the task."

# Run the agent
if __name__ == "__main__":
    result = run_agent(
        "The payment-service pod payment-service-7d4b8c-x2k9p seems to be crashing. "
        "Can you investigate and take appropriate action?"
    )

```

Expected Agent Behavior

```

--- Agent iteration 1 ---
☒ Calling: get_pod_status({"pod_name": "payment-service-7d4b8c-x2k9p"})
☒ Result: {"pod": "payment-service-7d4b8c-x2k9p", "status": "CrashLoopBackOff"...}

--- Agent iteration 2 ---
☒ Calling: get_pod_logs({"pod_name": "payment-service-7d4b8c-x2k9p", "previous": true})
☒ Result: {"lines": ["...OutOfMemoryError: Java heap space..."]}

--- Agent iteration 3 ---
☒ Calling: get_recent_deployments({"service_name": "payment-service"})
☒ Result: {"deployments": [{"version": "v2.4.1", "changes": "Added transaction caching..."}]}

--- Agent iteration 4 ---
☒ Calling: create_incident_ticket({"title": "payment-service OOMKilled after v2.4.1...", ...})
☒ Result: {"ticket_id": "INC0012345", "status": "created"}

☒ Agent response:
Investigation complete. Here's what I found:

```

****Root Cause:**** The payment-service pod is being OOMKilled (exit code 137). Memory usage peaked at 412Mi against a 256Mi limit. The issue started after deployment of v2.4.1 (45 minutes ago), which added "transaction caching for performance improvement" — this caching is likely consuming more memory than

the current limit allows.

****Actions Taken:****

1. Created incident ticket INC0012345
2. Recommended: Increase memory limit to 512Mi or rollback v2.4.1

****Immediate Recommendation:****

Rollback to v2.4.0 to restore service, then work with the development team to optimize the transaction cache memory usage before re-deploying v2.4.1 with a higher memory limit.

Explanation "The agent gathered evidence systematically — status, logs, recent deployments — then synthesized the information to identify the root cause (new deployment added caching that exceeded memory limits), took action (created a ticket), and provided a recommendation. It followed the same investigation process a senior SRE would."

PART 3: Designing Good Tool Definitions

Tool Description Quality Matters

Explanation "The model chooses which tool to use based on the tool's description. A vague description leads to wrong tool selection. A clear description leads to correct usage."

BAD TOOL DEFINITION:

```
{
  "name": "get_data",
  "description": "Gets some data" ❌ TERRIBLE. What data? When? Why?
}
```

GOOD TOOL DEFINITION:

```
{
  "name": "get_pod_status",
  "description": "Get the current status of a Kubernetes pod including its state, restart count, and resource usage. Use this when you need to check if a pod is healthy or diagnose pod issues."
}
```

Rules for tool descriptions:

1. **What it does** — "Get the current status of a Kubernetes pod"
2. **What it returns** — "including its state, restart count, and resource usage"
3. **When to use it** — "Use this when you need to check if a pod is healthy"
4. **Constraints** — "IMPORTANT: Only create tickets for ERROR or CRITICAL issues"

Parameter Schema Quality

BAD:

```
"properties": {  
  "name": {"type": "string"}  ❏ Which name? Pod name? Service name?  
}
```

GOOD:

```
"properties": {  
  "pod_name": {  
    "type": "string",  
    "description": "Full pod name (e.g., 'payment-service-7d4b8c-x2k9p')"  
  }  
}
```

Include example values in descriptions. The model uses examples to understand the expected format.

PART 4: The ReAct Pattern

Reasoning + Acting

Explanation "ReAct stands for Reasoning + Acting. Instead of just calling tools, the model explicitly writes out its reasoning before each action. This makes the agent more transparent and often more accurate."

WITHOUT ReAct (implicit reasoning):

- ❏ calls `get_pod_status()`
- ❏ calls `get_pod_logs()`
- ❏ calls `create_ticket()`

(The model's reasoning is hidden — you don't know WHY it chose each tool)

WITH ReAct (explicit reasoning):

THOUGHT: "The user reports a crashing pod. I should first check its status to understand the current state."

ACTION: `get_pod_status("payment-service-7d4b8c-x2k9p")`

OBSERVATION: Status is `CrashLoopBackOff`, `OOMKilled`, restart count 5

THOUGHT: "It's being `OOMKilled`. I should check logs to see what's consuming memory, and check recent deployments in case a code change caused this."

ACTION: `get_pod_logs("payment-service-7d4b8c-x2k9p", previous=true)`

OBSERVATION: Logs show `OutOfMemoryError` during cache initialization

THOUGHT: "The OOM happens during cache initialization. Let me check if there was a recent deployment that added caching."

ACTION: `get_recent_deployments("payment-service")`

OBSERVATION: v2.4.1 deployed 45 minutes ago, changes: "Added transaction caching"

THOUGHT: "Root cause identified: v2.4.1 introduced transaction caching that exceeds the 256Mi memory limit. This is a CRITICAL issue"

affecting the payment service. I should create an incident ticket."
ACTION: create_incident_ticket(...)

How to enable ReAct: Add to the system prompt:

Before each tool call, write a THOUGHT explaining:

- 1. What you know so far
- 2. What you need to find out
- 3. Why this specific tool call will help

After each tool result, write an OBSERVATION summarizing what you learned.

When to use ReAct vs simple tool calling:

Use ReAct	Skip ReAct
Complex investigations (multiple tools, complex reasoning)	Simple single-tool tasks
Debugging/transparency is important	High-volume automated processing (cost of extra tokens)
The agent needs to reason about which tool to call	The tool choice is obvious from the input

PART 5: Error Handling in Agents

What Can Go Wrong

Explanation "Agents add a new dimension of failure beyond what you've dealt with in prompts and RAG. Now you have tool execution failures, wrong tool selection, and infinite loops."

1. Tool Execution Failure

```
def execute_tool_safe(tool_name: str, tool_input: dict) -> str:
    """Execute a tool with error handling."""
    try:
        result = execute_tool(tool_name, tool_input)
        return result
    except TimeoutError:
        return json.dumps({
            "error": "Tool timed out after 30 seconds",
            "tool": tool_name,
            "suggestion": "Try again or use an alternative approach"
        })
    except Exception as e:
        return json.dumps({
            "error": str(e),
```

```

        "tool": tool_name,
        "suggestion": "This tool is currently unavailable"
    })

```

The model handles errors gracefully — it reads the error message and adapts its strategy.

2. Wrong Tool Selection The model calls a tool that doesn't fit the situation. Mitigated by clear tool descriptions and testing.

3. Infinite Loops

```

# ALWAYS set a max iteration limit
MAX_ITERATIONS = 10

# Track repeated tool calls
tool_call_history = []
for block in response.content:
    if block.type == "tool_use":
        call_key = f"{block.name}:{json.dumps(block.input)}"
        if call_key in tool_call_history:
            print("⚠ Agent is repeating a tool call — breaking loop")
            break
        tool_call_history.append(call_key)

```

4. Hallucinated Tool Calls The model invents a tool that doesn't exist. Your code should handle this:

```

if tool_name not in AVAILABLE_TOOLS:
    result = json.dumps({
        "error": f"Tool '{tool_name}' does not exist. Available tools: {list(AVAILABLE_TOOLS.keys())}"
    })

```

PART 6: Agent Guardrails

Keeping Agents Safe

Explanation "An agent without guardrails is a liability. It could create 1,000 tickets, delete production pods, or run up a massive API bill. Guardrails are non-negotiable."

Essential guardrails:

```

GUARDRAILS = {
    "max_iterations": 10,          # Prevent infinite loops
    "max_tokens_budget": 50000,    # Cost control
    "max_tool_calls": 20,          # Prevent excessive API calls
}

```

```

"require_confirmation": [      # Human approval for destructive actions
    "create_incident_ticket",
    "restart_pod",
    "rollback_deployment",
    "scale_deployment"
],
"read_only_mode": False,      # If True, only allow read tools
"allowed_namespaces": [      # Scope of access
    "production",
    "staging"
]
}

def check_guardrails(tool_name: str, tool_input: dict, iteration: int) -> tuple[bool, str]:
    """Check if a tool call is allowed."""
    if iteration >= GUARDRAILS["max_iterations"]:
        return False, "Maximum iterations reached"

    if tool_name in GUARDRAILS["require_confirmation"]:
        # In production, this would pause and ask the human
        print(f"⚠️ CONFIRMATION REQUIRED: Agent wants to call {tool_name}")
        print(f"  Parameters: {json.dumps(tool_input, indent=2)}")
        confirm = input("  Approve? (y/n): ")
        if confirm.lower() != 'y':
            return False, "Action rejected by operator"

    if GUARDRAILS["read_only_mode"] and tool_name in ["create_incident_ticket", "restart_pod"]:
        return False, "Read-only mode — write operations disabled"

    return True, "OK"

```

Explanation "Your observability agent will have a human-in-the-loop checkpoint for Sev1 incidents. The agent can investigate autonomously, but creating tickets and taking remediation actions for critical incidents requires human approval. This is the same principle as requiring a second approval for production deployments."

REFERENCE MATERIAL

1. Anthropic Tool Use Documentation — Official guide to tool calling with Claude. <https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview>
2. "Building Effective Agents" by Anthropic — Best practices for agent design. <https://www.anthropic.com/research/building-effective-agents>
3. ReAct Paper — "ReAct: Synergizing Reasoning and Acting in Language Models." The original paper. <https://arxiv.org/abs/2210.03629>